

[29] Survey of vector database management systems 총정리

저자: J. J. Pan, J. Wang, G. Li

학회/출판: The VLDB Journal, vol. 33, no. 5, 2024

페이지: p. 1591–1615

발행월: September 2024

자료형: 종합 서베이 논문

요약

벡터 데이터베이스 관리 시스템의 포괄적이고 최신의 서베이 논문이다. 이 논문은 벡터 DB의 탄생 배경, 발전 역사, 현재의 다양한 시스템들을 체계적으로 분류하고 분석한다. 벡터 DB의 핵심 기술 영역인 인덱싱, 근사 최근접 이웃 검색, 메타데이터 관리, 분산 처리, 그리고 최적화 기법들을 상세히 다룬다. VLDB 저널의 게재를 통해 데이터베이스 커뮤니티의 공식적 인정을 받은 이 서베이는, 벡터 DB 관련 최신 연구 동향을 정리하고, 해결되지 않은 문제들을 지적하며, 향후 연구 방향을 제시한다. 약 600 개 이상의 참고문헌을 포함하여, 벡터 DB 분야의 가장 종합적이고 신뢰할 수 있는 기술 참고자료이다.

상세분석

29.1 벡터 DB의 역사적 발전

등장 배경

전통 DB의 한계:

- 관계형 DB: 구조화된 데이터에 최적화, 벡터 데이터 처리 부실
- 문서 DB: 반구조화 데이터에 강하나 의미적 유사도 검색 미지원

AI 혁명의 영향:

- 딥러닝의 급속한 발전으로 특성 추출(feature extraction)이 표준화
- 대규모 텍스트 임베딩 모델의 등장 (Word2Vec, BERT, GPT 기반 임베딩)
- LLM 기반 애플리케이션의 외부 지식 베이스 필요성

벡터 DB의 필연성:

- 수백만 수억 차원의 벡터 데이터를 효율적으로 관리할 수 있는 전문 시스템의 부재
- 전통 DB에 벡터 지원 추가의 복잡성과 성능 한계

29.2 핵심 분류 체계

시스템 분류 관점

1. 전문 벡터 DB (Specialized Vector Databases):

- Milvus, Weaviate, Vespa, Pinecone 등
- 벡터 검색에 최적화
- 일반 데이터베이스 기능은 제한적

2. 벡터 DB 확장 (Vector Database Extensions):

- PostgreSQL pgvector: 기존 RDBMS 에 벡터 지원 추가
- DuckDB-VSS: 컬럼 기반 DB 에 벡터 검색 추가
- Elasticsearch: 검색 엔진에 벡터 검색 추가

3. 하이브리드 벡터 DB (Hybrid Vector-Relational):

- 벡터와 정형 데이터를 동시에 효율적으로 처리
- 복합 쿼리 지원 (스칼라 필터 + 벡터 검색)

4. 그래프 벡터 DB (Graph Vector Databases):

- Neo4j: 그래프 관계와 벡터 유사도의 결합
- 복잡한 관계 데이터의 표현과 검색

29.3 주요 기술 영역

1. 인덱싱 기법

메모리 기반 인덱싱:

- HNSW (Hierarchical Navigable Small World): 다층 그래프 구조, 빠른 검색
- IVF (Inverted File Index): 클러스터 기반, 효율적 메모리 사용
- LSH (Locality Sensitive Hashing): 확률적 기법, 병렬 처리 용이

디스크 기반 인덱싱:

- DiskANN: 메모리 기반 그래프와 디스크 저장의 결합
- Product Quantization: 벡터 압축으로 저장소 절감

하이브리드 인덱싱:

- 메모리와 디스크를 모두 활용하는 적응형 인덱싱

2. 근사 최근접 이웃 (ANN) 검색 알고리즘

정확도-성능 트레이드오프:

- Recall@k: 상위 k 개 결과의 정확도 (일반적으로 95% 이상 목표)
- Query Latency: 밀리초 단위 응답 시간
- Throughput: 초당 처리 쿼리 수

주요 메트릭:

- $\text{Recall} = (\text{실제 최근접 } k \text{ 개 중 검색된 수}) / k$
- 일반적으로 Recall 95%에서 지연시간 10-100ms 달성

3. 메타데이터 관리

메타데이터의 중요성:

- 벡터 자체는 의미 정보를 담지만, 문서 출처, 카테고리, 타임스탬프 등은 벡터와 분리 필요
- 필터링 조건과 벡터 유사도 검색의 조합이 필수

저장소 접근법:

- 별도 메타데이터 DB: 빠른 필터링, 추가 인덱싱
- 벡터 저장소 내 메타데이터: 간단하나 필터링 성능 저하

복합 쿼리 처리:

- WHERE 절과 벡터 유사도 조건의 동시 처리
- 성능 최적화: 필터 우선 vs 벡터 검색 우선 선택

4. 분산 벡터 DB

샤딩 전략:

- 범위 기반 샤딩: 메타데이터 범위로 분할
- 해시 기반 샤딩: 벡터 ID의 해시값으로 분할
- 의미적 샤딩: 유사 벡터를 같은 샤드에 배치 (성능 향상)

일관성 보장:

- 강한 일관성: 모든 복제본이 동일 (성능 감소)
- 최종 일관성: 모든 복제본이 결국 동일 (성능 향상)

분산 쿼리 처리:

- 브로드캐스트: 모든 샤드에 쿼리 전송 후 결과 병합 (정확하나 느림)
- 샤드 선택: 관련 샤드만 쿼리 (빠르나 복잡)

5. 동적 업데이트

삽입/삭제의 도전:

- 기존 인덱스 구조 변경 최소화
- 실시간 쓰기 성능 보장

로그 기반 아키텍처:

- Write-ahead Log (WAL): 데이터 손실 방지
- 메모리 버퍼: 최근 수정 사항 빠른 접근
- 주기적 플러시: 저장소로의 데이터 이전

29.4 주요 벡터 DB 시스템 비교

개방형 vs 상용형

개방형 (Open-source):

- Milvus: 클라우드 네이티브, 높은 확장성
- Weaviate: GraphQL API, 스키마 유연성
- Chroma: 경량, 개발 친화적

상용형 (Commercial):

- Pinecone: 완전 관리형, 높은 가용성
- Vespa: Yahoo 의 고성능 시스템

배포 모델

자관리형 (Self-managed):

- 높은 커스터마이징 가능
- 운영 복잡성 증가

클라우드 관리형 (Cloud-managed):

- 운영 부담 감소
- 비용 증가

29.5 본 논문과의 관계

Exqutor 은 텍스트 쿼리를 벡터 검색으로 변환하는 하이브리드 검색 시스템이다. 이 종합 서베이는 Exqutor 이 의존하는 벡터 DB 기술의 모든 측면을 다룬다:

1. **인덱싱 선택:** Exqutor 이 사용할 인덱싱 기법의 설계 원칙 제공
2. **메타데이터 필터링:** 텍스트 쿼리와 메타데이터 필터의 조합 처리 방식
3. **분산 아키텍처:** Exqutor 의 대규모 환경 확장 전략
4. **최적화 기법:** 성능과 정확도 간 트레이드오프 관리

또한 이 서베이는 Exqutor 의 위치를 벡터 DB 생태계 내에서 명확히 하는 데 도움이 된다 - Exqutor 은 단순 벡터 DB 가 아닌 텍스트-벡터 변환과 검색을 통합한 더 높은 수준의 시스템이다.

추가 제기 문제

1. **벡터 DB의 표준화:** 다양한 벡터 DB 시스템 간에 호환성 있는 API 나 데이터 포맷 표준화는 가능한가?
2. **인덱싱 기법의 선택 자동화:** 주어진 데이터 특성과 워크로드에 대해 최적의 인덱싱 기법을 자동으로 선택하는 인텔리전트 시스템은?
3. **메타데이터-벡터 조인의 최적화:** 복잡한 메타데이터 필터 조건과 벡터 유사도를 결합할 때, 옵티마이저의 최적 실행 계획 수립은?
4. **벡터 임베딩의 버전 관리:** 새로운 임베딩 모델이 등장했을 때 기존 벡터 데이터의 점진적 마이그레이션 전략은?
5. **실시간 성능 특성 변화:** 시간이 흐르면서 데이터 특성이 변할 때, 인덱스와 파라미터를 자동으로 적응시키는 기능은?
6. **벡터 DB 선택 프레임워크:** 기업이 자신의 요구사항에 맞는 벡터 DB 를 선택할 수 있는 의사결정 프레임워크 수립?